```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

```
def cost_function(x):
    return x**2 + 4*x + 4
```

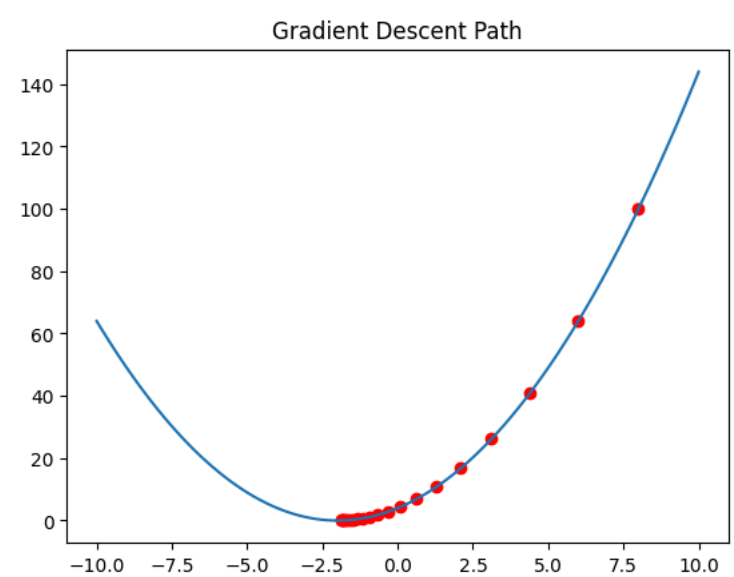
```
def derivative(x):
    return 2*x + 4
```

```
x = np.linspace(-10,10,100)
y = cost_function(x)

plt.plot(x,y)
plt.title("Cost Function")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.show()
```

```
learning_rate = 0.1
x_current = 8
iterations = 20

x_history = [x_current]
y_history = [cost_function(x_current)]

for i in range(iterations):

    gradient = derivative(x_current)
    x_current = x_current - learning_rate*gradient

    x_history.append(x_current)
    y_history.append(cost_function(x_current))

print("Minimum value occurs at:", x_current)
```

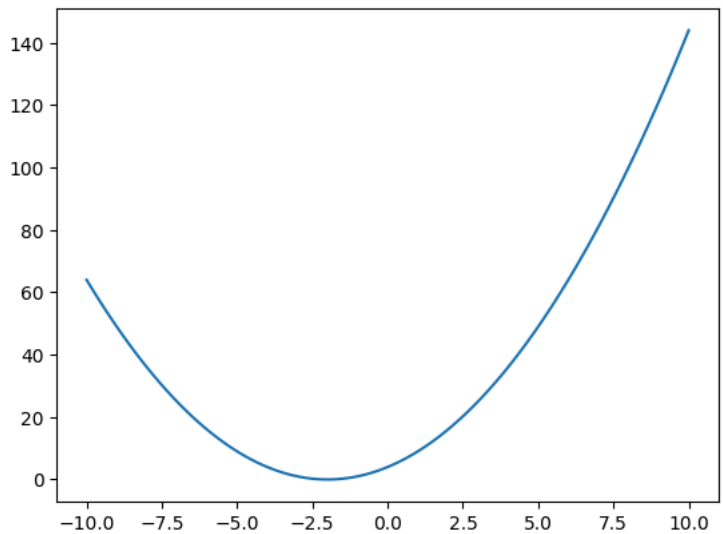
```
Minimum value occurs at: -1.8847078495393152
```

```
plt.plot(x,y)

plt.scatter(x_history,y_history,color='red')

plt.title("Gradient Descent Path")
plt.show()
```




```
fig, ax = plt.subplots()

ax.plot(x,y)
```

```
ax.plot(x,y)
point, = ax.plot([], [], 'ro')

def update(frame):

    point.set_data(x_history[frame], y_history[frame])

    return point,

ani = FuncAnimation(fig, update, frames=len(x_history), interval=500)

plt.show()
```



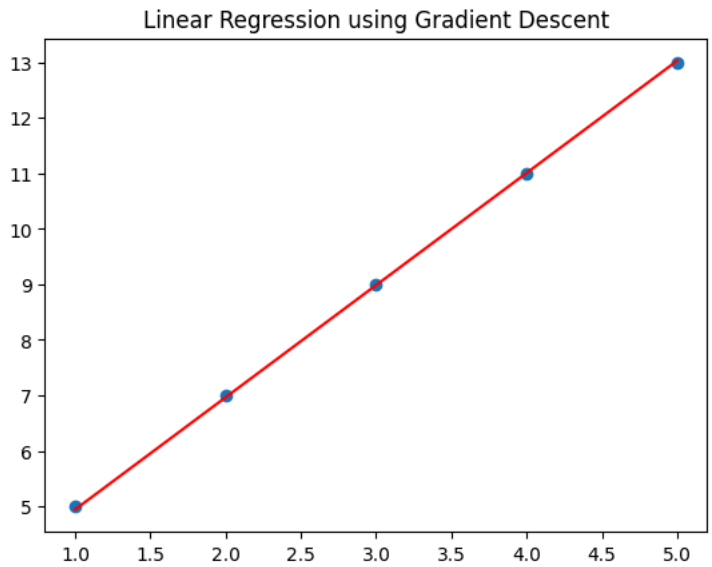

```
X = np.array([1,2,3,4,5])
Y = np.array([5,7,9,11,13])
```

```
w = 0
b = 0
learning_rate = 0.01
epochs = 1000
n = len(X)
```

```
for i in range(epochs):

    y_pred = w*X + b

    dw = -(2/n) * np.sum(X*(Y - y_pred))
    db = -(2/n) * np.sum(Y - y_pred)

    w = w - learning_rate*dw
    b = b - learning_rate*db

print("Weight:",w)
print("Bias:",b)
```

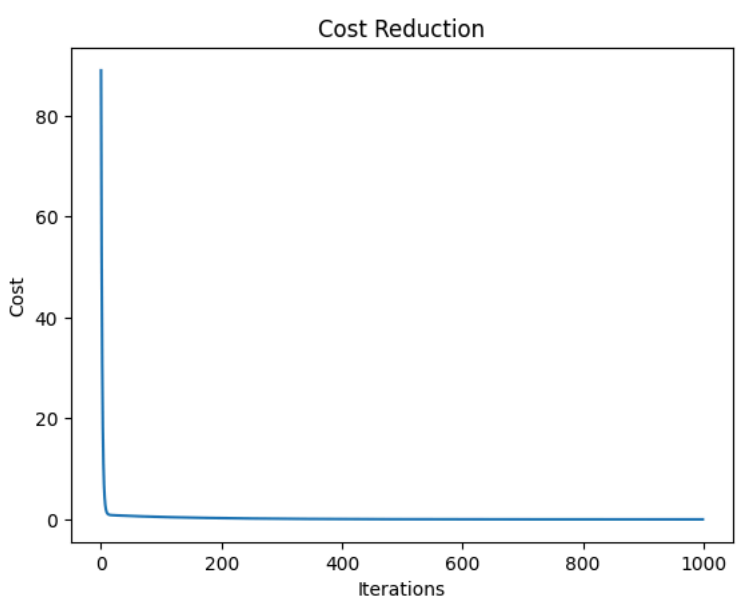
```
Weight: 2.021281045682893
Bias: 2.923168672645527
```

```
y_pred = w*X + b

plt.scatter(X,Y)
plt.plot(X,y_pred,color='red')

plt.title("Linear Regression using Gradient Descent")
plt.show()
```




```
costs = []

w = 0
b = 0

for i in range(epochs):

    y_pred = w*X + b

    cost = np.mean((Y-y_pred)**2)
    costs.append(cost)

    dw = -(2/n) * np.sum(X*(Y - y_pred))
    db = -(2/n) * np.sum(Y - y_pred)

    w = w - learning_rate*dw
    b = b - learning_rate*db
```

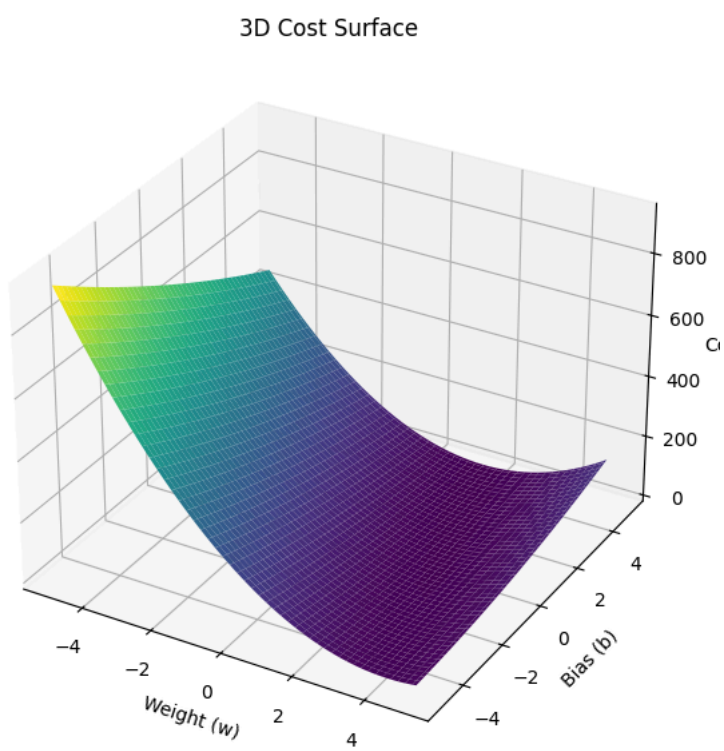
```
plt.plot(costs)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Cost Reduction")
plt.show()
```



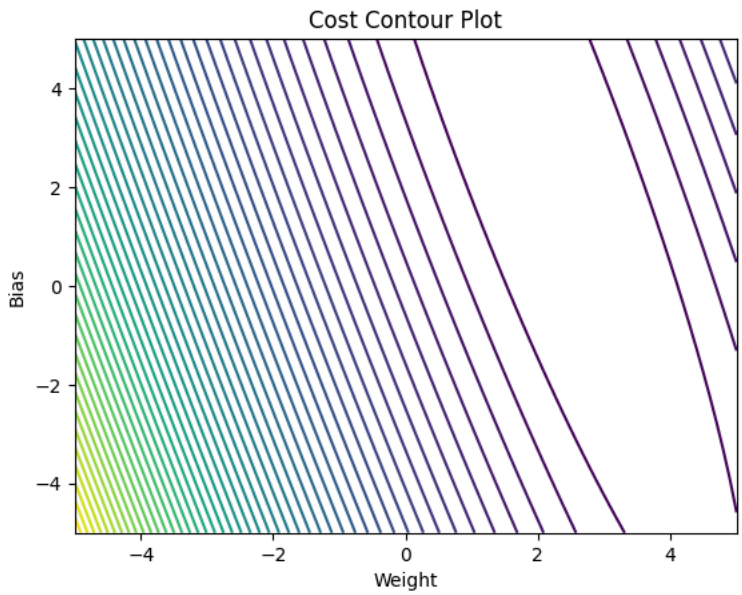

```
from mpl_toolkits.mplot3d import Axes3D
```

```
w_vals = np.linspace(-5,5,100)
b_vals = np.linspace(-5,5,100)

W, B = np.meshgrid(w_vals, b_vals)

Z = np.zeros(W.shape)

for i in range(W.shape[0]):
    for j in range(W.shape[1]):

        y_pred = W[i,j]*X + B[i,j]
        Z[i,j] = np.mean((Y - y_pred)**2)
```

```
fig = plt.figure(figsize=(10,7))

ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(W, B, Z, cmap='viridis')

ax.set_xlabel("Weight (w)")
ax.set_ylabel("Bias (b)")
ax.set_zlabel("Cost")

plt.title("3D Cost Surface")

plt.show()
```

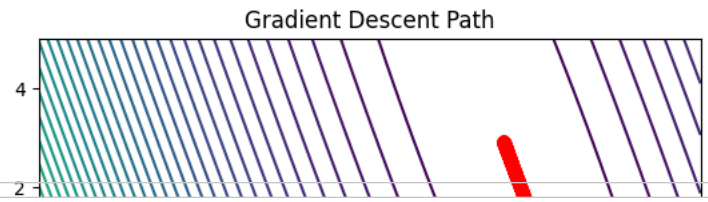


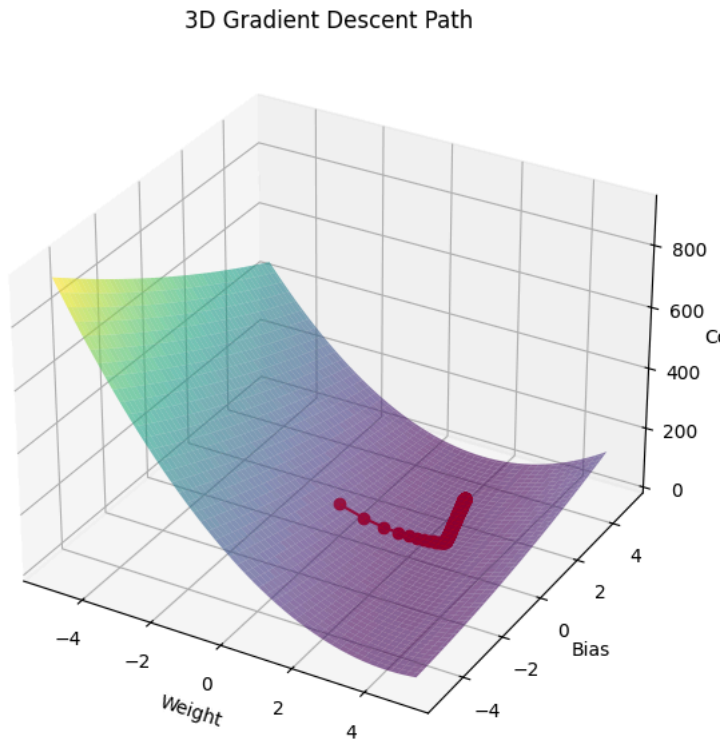

```
plt.contour(W, B, Z, 50)

plt.xlabel("Weight")
plt.ylabel("Bias")

plt.title("Cost Contour Plot")

plt.show()
```




```
w = 0
b = 0

w_path = []
b_path = []
cost_path = []

for i in range(epochs):

    y_pred = w*X + b

    cost = np.mean((Y - y_pred)**2)

    w_path.append(w)
    b_path.append(b)
    cost_path.append(cost)

    dw = -(2/n) * np.sum(X*(Y - y_pred))
    db = -(2/n) * np.sum(Y - y_pred)

    w = w - learning_rate*dw
    b = b - learning_rate*db
```

```
plt.contour(W,B,Z,50)

plt.plot(w_path,b_path,'r-o')

plt.xlabel("Weight")
plt.ylabel("Bias")

plt.title("Gradient Descent Path")

plt.show()
```




```
fig = plt.figure(figsize=(10,7))

ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(W,B,Z, cmap='viridis', alpha=0.6)

ax.plot(w_path,b_path,cost_path,color='red',marker='o')

ax.set_xlabel("Weight")
ax.set_ylabel("Bias")
ax.set_zlabel("Cost")

plt.title("3D Gradient Descent Path")

plt.show()
```